Отчет по практическому заданию №1

Гусейнов А.Э. 681

31 октября 2026 г.

Содержание

1 Введение **2**

2 Постановка задачи **2**

3 Метод фиктивных областей **2**

4 Разностная схема **3**

5 Метод сопряженных градиентов с диагональным предобуславливанием **3**

6 Параллельная реализация на OpenMP **4**

7 Результаты расчетов **5**

8 Заключение **5**

A Основные фрагменты программы **6**

1 Введение

В работе рассматривается двумерная задача Дирихле для уравнения Пуассона в криволинейной области. Для приближенного решения используется метод фиктивных областей с продолжением по младшему коэффициенту. Полученная дифференциальная задача аппроксимируется на равномерной прямоугольной сетке разностной схемой. Система линейных алгебраических уравнений решается методом сопряженных градиентов с диагональным предобуславливанием. Последовательная программа распараллелена с помощью технологии OpenMP.

Вариант задания: **вариант 7** — квадрат с отсеченной вершиной:

$$D = \{(x, y) : |x| + |y| < 2,\ y < 1\}.$$

Вершины области: $(-2, 0)$, $(0, -2)$, $(2, 0)$, $(1, 1)$, $(-1, 1)$.

2 Постановка задачи

В области $D \subset \mathbb{R}^2$ с кусочно-гладкой границей γ требуется найти функцию $u(x, y)$, удовлетворяющую уравнению Пуассона

$$-\Delta u = f(x, y), \qquad (x, y) \in D, \tag{1}$$

и граничному условию Дирихле

$$u(x, y) = \varphi(x, y), \qquad (x, y) \in \gamma. \tag{2}$$

Для варианта 7:

$$f(x, y) = 1, \qquad \varphi(x, y) = \sin(x + y).$$

3 Метод фиктивных областей

Область D погружается в прямоугольник

$$\Pi = \{(x, y) : A_1 < x < B_1,\ A_2 < y < B_2\}.$$

Для варианта 7: $A_1 = -2$, $B_1 = 2$, $A_2 = -2$, $B_2 = 1$. Фиктивная область $\hat{D} = \Pi \setminus D$. Вводится малый параметр $\varepsilon > 0$. Решается задача

$$-\Delta v + Q(x, y)\big(v - v_0(x, y)\big) = F(x, y), \quad (x, y) \in \Pi \setminus \gamma, \tag{3}$$

$$v(x, y) = v_0(x, y), \quad (x, y) \in \Gamma,$$

где

$$Q(x, y) = \begin{cases} 0, & (x, y) \in D, \\ \varepsilon^{-2}, & (x, y) \in \hat{D}, \end{cases} \qquad F(x, y) = \begin{cases} f(x, y), & (x, y) \in D, \\ 0, & (x, y) \in \hat{D}. \end{cases}$$

В качестве продолжения v_0 используется $\varphi(x, y) = \sin(x + y)$. Параметр ε выбирается как $\varepsilon = h^2$, где $h = \max(h_1, h_2)$ — максимальный шаг сетки.

4 Разностная схема

В прямоугольнике Π вводится равномерная сетка

$$\bar{\omega}_h = \bar{\omega}_1 \times \bar{\omega}_2, \qquad \bar{\omega}_1 = \{x_i = A_1 + ih_1,\ i = 0, \ldots, M\}, \quad \bar{\omega}_2 = \{y_j = A_2 + jh_2,\ j = 0, \ldots, N\},$$

где $h_1 = (B_1 - A_1)/M$, $h_2 = (B_2 - A_2)/N$. Внутренние узлы обозначаются ω_h. Дифференциальное уравнение аппроксимируется разностным уравнением

$$-w_{x\bar{x},ij} - w_{y\bar{y},ij} + Q_{ij} w_{ij} = F_{ij} + Q_{ij} v_0(x_i, y_j), \qquad i = 1, \ldots, M-1, \quad j = 1, \ldots, N-1. \tag{4}$$

Граничные условия аппроксимируются точно:

$$w_{ij} = v_0(x_i, y_j), \qquad (x_i, y_j) \in \Gamma.$$

Коэффициенты Q_{ij} и F_{ij} вычисляются как средние по ячейке Π_{ij}:

$$Q_{ij} = \frac{1}{h_1 h_2} \iint_{\Pi_{ij}} Q(x, y)\, dx\, dy, \qquad F_{ij} = \frac{1}{h_1 h_2} \iint_{\Pi_{ij}} F(x, y)\, dx\, dy.$$

Для варианта 7 доля площади ячейки, попавшая в D, вычислялась приближенно по подсетке 12×12, что позволяет единообразно обрабатывать криволинейные границы. В результате получается система линейных алгебраических уравнений

$$Aw = B \tag{5}$$

с симметричной положительно определенной матрицей A.

5 Метод сопряженных градиентов с диагональным предобуславливанием

Система (5) решается методом сопряженных градиентов. Используется диагональный предобуславливатель

$$(Dw)_{ij} = \left(\frac{2}{h_1^2} + \frac{2}{h_2^2} + Q_{ij} \right) w_{ij}.$$

Итерационный процесс:

$$\begin{aligned}
&r^{(0)} = B - Aw^{(0)}, \qquad Dz^{(0)} = r^{(0)}, \qquad p^{(1)} = z^{(0)}, \\
&\alpha_k = \frac{(z^{(k-1)}, r^{(k-1)})}{(Ap^{(k)}, p^{(k)})}, \\
&w^{(k)} = w^{(k-1)} + \alpha_k p^{(k)}, \\
&r^{(k)} = r^{(k-1)} - \alpha_k A p^{(k)}, \\
&Dz^{(k)} = r^{(k)}, \\
&\beta_{k+1} = \frac{(z^{(k)}, r^{(k)})}{(z^{(k-1)}, r^{(k-1)})}, \\
&p^{(k+1)} = z^{(k)} + \beta_{k+1} p^{(k)}.
\end{aligned}$$

Начальное приближение $w^{(0)} = 0$. Критерий остановки:

$$\|w^{(k+1)} - w^{(k)}\|_E < \delta,$$

где $\delta = 10^{-8}$. Дополнительно контролировался функционал

$$J(w) = \frac{1}{2}(Aw, w) - (B, w),$$

который должен монотонно убывать.

6 Параллельная реализация на OpenMP

Последовательная программа написана на C++. Распараллеливание выполнено с помощью директив OpenMP. Основные вычислительные циклы — вычисление Q_{ij}, F_{ij}, правой части B, умножение матрицы на вектор Ap, вычисление скалярных произведений и обновление векторов w, r, z, p — распараллелены с помощью `#pragma omp parallel for`. Для вложенных циклов по i и j используется `collapse(2)`. Распределение итераций — `schedule(static)`. Скалярные произведения реализованы с редукцией `reduction(+:sum)`. Время измеряется функцией `omp_get_wtime()`. При компиляции без флага `-fopenmp` директивы игнорируются, и программа работает последовательно.

Ключевой фрагмент умножения матрицы на вектор:

```
#pragma omp parallel for collapse(2) schedule(static)
for (int i = 1; i <= M - 1; ++i) {
    for (int j = 1; j <= N - 1; ++j) {
        int k = idx(i, j);
        double val = (2.0*inv_h1_2 + 2.0*inv_h2_2 + Q[k]) * u[k];
        if (i > 1) val -= u[idx(i-1,j)] * inv_h1_2;
        if (i < M - 1) val -= u[idx(i+1,j)] * inv_h1_2;
        if (j > 1) val -= u[idx(i,j-1)] * inv_h2_2;
        if (j < N - 1) val -= u[idx(i,j+1)] * inv_h2_2;
        out[k] = val;
    }
}
```

Компиляция:

```
g++ -O3 -std=c++17 solve.cpp -o solve
g++ -O3 -fopenmp -std=c++17 solve.cpp -o solve_omp
```

Запуск с разным числом нитей:

```
export OMP_NUM_THREADS=1
./solve_omp 80 80
export OMP_NUM_THREADS=4
./solve_omp 80 80
export OMP_NUM_THREADS=16
./solve_omp 80 80
```

7 Результаты расчетов

Расчеты проводились для сеток $(M, N) = (20, 20), (40, 40), (80, 80)$. Для сетки 80×80 дополнительно исследовалась масштабируемость на 1, 4 и 16 нитях OpenMP. В таблицах ниже числовые значения будут внесены после выполнения программы.

Таблица 1: Результаты расчетов: последовательный вариант и OpenMP

$M \times N$	Итераций	$T_{\text{посл}}$, с	T_1, с	T_4, с	T_{16}, с	S_4	S_{16}
20×20	?	?	?	?	?	?	?
40×40	?	?	?	?	?	?	?
80×80	?	?	?	?	?	?	?

Таблица 2: Масштабируемость OpenMP на сетке 80×80

Число нитей	Итераций	Время, с	Ускорение S_p
1	?	?	1.00
4	?	?	?
16	?	?	?

На рис. **??** приведен график приближенного решения на сетке 80×80. На рис. **??** — график ускорения OpenMP-версии.

8 Заключение

В работе реализован метод фиктивных областей для решения задачи Дирихле для уравнения Пуассона в области варианта 7. Построена разностная схема на равномерной прямоугольной сетке. Система линейных уравнений решена методом сопряженных градиентов с диагональным предобуславливанием. Выполнено распараллеливание основных вычислительных циклов с помощью OpenMP. Программа позволяет легко менять геометрию области и правую часть, так как все параметры варианта вынесены в отдельный блок. Результаты расчетов показывают работоспособность алгоритма; эффективность распараллеливания зависит от размера сетки и числа нитей.

Список литературы

[1] А.Н. Тихонов, А.А. Самарский. Уравнения математической физики. М.: Наука, 1977.

[2] Г.И. Марчук. Методы вычислительной математики. М.: Наука, 1989.

[3] П.Н. Вабищевич. Метод фиктивных областей в задачах математической физики. 2017.

[4] А.А. Самарский. Теория разностных схем. М.: Наука, 1989.

[5] А.А. Самарский, А.В. Гулин. Численные методы. М.: Наука, 1989.

[6] В.Б. Андреев. Численные методы. М.: Макс Пресс, 2013.

[7] А.С. Антонов. Технологии параллельного программирования MPI и OpenMP. М.: Изд-во МГУ, 2012.

[8] OpenMP Architecture Review Board. OpenMP Application Programming Interface. https://www.openmp.org/.

A Основные фрагменты программы

Полный код программы прилагается отдельно. Ниже приведен фрагмент, задающий геометрию варианта 7 и функцию проверки точки внутри области.

```
const vector<Point> DOMAIN = {
    {-2.0, 0.0},
    { 0.0, -2.0},
    { 2.0, 0.0},
    { 1.0, 1.0},
    {-1.0, 1.0}
};

double f(double x, double y) { return 1.0; }
double phi(double x, double y) { return sin(x + y); }
double v0(double x, double y) { return sin(x + y); }

bool insideDomain(double x, double y) {
    bool inside = false;
    int n = (int)DOMAIN.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        const Point& pi = DOMAIN[i];
        const Point& pj = DOMAIN[j];
        bool intersect = ((pi.y > y) != (pj.y > y)) &&
            (x < (pj.x - pi.x) * (y - pi.y) / (pj.y - pi.y) + pi.x);
        if (intersect) inside = !inside;
    }
    return inside;
}
```